LAPORAN

WORKSHOP SISTEM INFORMASI WEB SERVER (TIF330805)

SEMESTER 3



API CRUD DENGAN PHP NATIVE, OOP, DAN MVC

Disusun oleh :

Mochammad Syifaul Qulub (E41250219)

PROGRAM STUDI DILUAR KAMPUS UTAMA (PSDKU)
TEKNIK INFORMATIKA KAB. NGANJUK
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI JEMBER

TAHUN 2026

BAB 1. PENDAHULUAN

1.1 Latar Belakang

Application Programming Interface (API) merupakan antarmuka yang memungkinkan suatu aplikasi berkomunikasi dan bertukar data dengan aplikasi lain. Dalam pengembangan aplikasi web, API berperan penting dalam menyediakan layanan pengelolaan data yang dapat diakses melalui permintaan HTTP.

Pada praktikum Acara 17 mata kuliah Workshop Sistem Informasi Web Server, dilakukan pengembangan API CRUD menggunakan PHP Native dengan menerapkan konsep Object-Oriented Programming (OOP) dan pola arsitektur Model-View-Controller (MVC). Sistem menggunakan database MySQL sebagai tempat penyimpanan data mahasiswa serta PDO dan prepared statement untuk menjalankan perintah SQL.

API yang dikembangkan memiliki fungsi untuk menampilkan seluruh data mahasiswa, menampilkan data berdasarkan ID, menambahkan data, memperbarui data, dan menghapus data. Sistem juga dilengkapi validasi untuk mencegah penggunaan Nomor Induk Mahasiswa (NIM) yang sama. Melalui praktikum ini, mahasiswa diharapkan mampu memahami proses pembuatan API yang terstruktur dan mudah dikembangkan.

1.2 Rumusan Masalah

1. Bagaimana membangun API CRUD menggunakan PHP Native dengan konsep OOP dan MVC?
2. Bagaimana menghubungkan aplikasi PHP dengan database MySQL menggunakan PDO dan prepared statement?
3. Bagaimana menerapkan autoloader, front controller, dan router dalam pengelolaan endpoint API?
4. Bagaimana mengimplementasikan dan menguji operasi GET, POST, PUT, dan DELETE?
5. Bagaimana menerapkan validasi untuk mencegah penggunaan NIM duplikat?

1.3 Tujuan Praktikum

1. Memahami penerapan konsep OOP dan pola MVC dalam pengembangan API.
2. Membuat API untuk mengelola data mahasiswa menggunakan PHP Native.
3. Menerapkan PDO dan prepared statement untuk mengakses database MySQL.
4. Memahami penggunaan autoloader, front controller, dan router.
5. Mengimplementasikan operasi CRUD melalui metode HTTP.
6. Menguji fungsi API dan menerapkan validasi NIM duplikat.

1.4 Manfaat Praktikum

Praktikum ini memberikan pemahaman mengenai pengembangan API menggunakan PHP Native, penerapan konsep OOP dan MVC, serta pengelolaan database menggunakan PDO. Selain itu, praktikum ini memberikan pengalaman dalam mengimplementasikan operasi CRUD, mengatur alur permintaan melalui router, dan melakukan pengujian API menggunakan browser serta PowerShell.

BAB 2. LANDASAN TEORI

2.1 Application Programming Interface (API)

Application Programming Interface (API) merupakan sekumpulan aturan yang memungkinkan aplikasi saling berkomunikasi dan bertukar data. API dapat menerima permintaan dari klien, memproses permintaan tersebut, dan mengembalikan respons sesuai hasil pemrosesan.

Pada praktikum ini, API digunakan untuk mengelola data mahasiswa dan mengembalikan hasil dalam format JSON (JavaScript Object Notation). Format JSON dipilih karena sederhana dan mudah digunakan oleh berbagai aplikasi.

2.2 REST dan HTTP Method

REST (Representational State Transfer) merupakan gaya arsitektur yang digunakan dalam pengembangan layanan berbasis sumber daya. Pada API, HTTP method digunakan untuk menentukan operasi yang akan dilakukan.

Metode HTTP yang diterapkan dalam praktikum ini adalah:

1. GET, digunakan untuk mengambil data mahasiswa.
2. POST, digunakan untuk menambahkan data mahasiswa baru.
3. PUT, digunakan untuk memperbarui data mahasiswa berdasarkan ID.
4. DELETE, digunakan untuk menghapus data mahasiswa berdasarkan ID.

Penggunaan metode tersebut memungkinkan pengelolaan data dilakukan melalui endpoint API tanpa harus menggunakan formulir pada halaman web.

2.3 PHP Native dan Object-Oriented Programming (OOP)

PHP Native merupakan pengembangan aplikasi menggunakan bahasa PHP tanpa bergantung pada framework utama. PHP dapat digunakan untuk menerima permintaan HTTP, menjalankan logika program, mengakses database, dan menghasilkan respons JSON.

Object-Oriented Programming (OOP) adalah paradigma pemrograman yang menggunakan kelas dan objek untuk menyusun program. Pada praktikum ini, OOP diterapkan melalui kelas Database, Mahasiswa, Router, dan

MahasiswaController. Pembagian kelas tersebut membuat kode lebih terorganisasi dan memudahkan pemeliharaan program.

2.4 Model-View-Controller (MVC)

Model-View-Controller (MVC) merupakan pola arsitektur yang memisahkan aplikasi menjadi tiga bagian utama.

1. Model bertanggung jawab mengelola data dan berkomunikasi dengan database.
2. View bertanggung jawab menyajikan tampilan kepada pengguna.
3. Controller bertanggung jawab memproses permintaan dan menghubungkan alur kerja antara model dengan keluaran aplikasi.

Pada aplikasi API ini, bagian Model diterapkan melalui kelas Mahasiswa, sedangkan Controller diterapkan melalui MahasiswaController. Karena aplikasi berfokus pada layanan API, hasil pemrosesan dikembalikan dalam format JSON, bukan melalui halaman HTML.

2.5 PDO dan Prepared Statement

PHP Data Objects (PDO) merupakan antarmuka PHP untuk mengakses berbagai jenis database. PDO mendukung penggunaan prepared statement sehingga query dapat disiapkan terlebih dahulu dan nilai input diberikan melalui parameter.

Prepared statement membantu mengurangi risiko SQL injection karena nilai input tidak langsung digabungkan ke dalam teks query SQL. Dalam praktikum ini, PDO digunakan untuk menjalankan operasi pengambilan, penambahan, pembaruan, dan penghapusan data mahasiswa.

2.6 Autoloader

Autoloader merupakan mekanisme untuk memuat file kelas secara otomatis ketika kelas tersebut diperlukan. PHP menyediakan fungsi spl_autoload_register() untuk mendaftarkan fungsi autoload.

Pada praktikum ini, autoloader ditempatkan pada file app/core/autoload.php. File tersebut mencari kelas yang diperlukan pada direktori konfigurasi, core, model, dan controller sehingga setiap kelas tidak perlu dipanggil secara manual satu per satu.

2.7 Front Controller dan Router

Front controller merupakan satu titik masuk utama yang menerima permintaan dari klien sebelum permintaan diproses oleh bagian aplikasi lainnya. Pada proyek ini, file public/index.php berfungsi sebagai front controller.

Router bertugas menentukan controller dan metode yang harus dijalankan berdasarkan endpoint serta HTTP method yang diterima. Dengan adanya router, permintaan terhadap endpoint mahasiswa dapat diarahkan ke fungsi yang sesuai, seperti pengambilan data, penambahan, pembaruan, atau penghapusan data.

BAB 3. METODOLOGI DAN IMPLEMENTASI

3.1 Alat dan Bahan

Alat dan bahan yang digunakan dalam praktikum ini meliputi:

1. Laptop atau komputer dengan sistem operasi Windows.
2. Laragon sebagai lingkungan server lokal.
3. PHP untuk menjalankan aplikasi.
4. MySQL/MariaDB sebagai sistem manajemen database.
5. phpMyAdmin untuk mengelola database.
6. Visual Studio Code sebagai editor kode program.
7. Google Chrome untuk mengakses endpoint GET.
8. PowerShell untuk menguji permintaan API.

3.2 Struktur Direktori Proyek

Proyek dibuat pada folder ACARA 17 API CRUD MVC yang berada di dalam direktori BKPM WORKSHOP SISTEM INFORMASI WEB SERVER.

Struktur direktori proyek adalah sebagai berikut.

```
ACARA 17 API CRUD MVC/
├── app/
│   ├── config/
│   │   └── Database.php
│   ├── core/
│   │   ├── autoload.php
│   │   └── Router.php
│   ├── models/
│   │   └── Mahasiswa.php
│   └── controllers/
│       └── MahasiswaController.php
├── public/
│   ├── index.php
│   └── .htaccess
```

└── database.sql

Struktur tersebut membagi program berdasarkan tanggung jawabnya. Direktori config berisi konfigurasi database, core berisi autoloader dan router, models berisi pengelolaan data, controllers berisi logika pemrosesan permintaan, sedangkan public menjadi direktori utama yang diakses melalui server web.

3.3 Perancangan Database

Database yang digunakan pada praktikum ini bernama acara 17. Database tersebut memiliki tabel mahasiswa untuk menyimpan data mahasiswa.
Struktur tabel mahasiswa terdiri atas kolom berikut.

Nama Kolom	Tipe Data	Keterangan
id	INT	Primary key dan auto increment
nim	VARCHAR(15)	Nomor induk mahasiswa dan bersifat unik
nama	VARCHAR(100)	Nama mahasiswa
prodi	VARCHAR(50)	Program studi
created_at	TIMESTAMP	Waktu pembuatan data

Kolom id digunakan sebagai identitas unik setiap data mahasiswa. Kolom nim memiliki batasan UNIQUE untuk mencegah penyimpanan NIM yang sama. Data awal yang digunakan dalam pengujian terdiri atas Budi Santoso, Siti Aminah, dan Andi Pratama.

3.4 Implementasi Program

3.4.1 Implementasi Koneksi Database

Koneksi database diimplementasikan pada file app/config/Database.php. Kelas Database menggunakan PDO untuk menghubungkan aplikasi PHP dengan MySQL. Konfigurasi koneksi mencakup host lokal, nama database, username, dan password.

Koneksi dibuat menggunakan metode statis getConnection() sehingga kelas lain dapat menggunakan koneksi yang sama. PDO juga dikonfigurasi untuk

menghasilkan exception ketika terjadi kesalahan serta mengembalikan hasil query dalam bentuk array asosiatif.

3.4.2 Implementasi Model Mahasiswa

Model Mahasiswa terdapat pada file app/models/Mahasiswa.php. Kelas ini bertanggung jawab menjalankan query untuk mengelola data mahasiswa.
Fungsi yang diterapkan meliputi:

1. getAll() untuk mengambil seluruh data mahasiswa.
2. find() untuk mengambil data berdasarkan ID.
3. create() untuk menambahkan data mahasiswa.
4. update() untuk memperbarui data mahasiswa.
5. delete() untuk menghapus data mahasiswa.

Seluruh operasi database menggunakan PDO dan prepared statement untuk mengelola parameter query.

3.4.3 Implementasi MahasiswaController

Controller terdapat pada file app/controllers/MahasiswaController.php. Controller bertugas menerima dan memproses permintaan, melakukan validasi input, memanggil model, serta mengembalikan respons dalam format JSON.

Field yang wajib diisi pada operasi penambahan dan pembaruan data adalah NIM, nama, dan program studi. Controller juga menangani ID yang tidak valid, data yang tidak ditemukan, format JSON yang salah, serta NIM yang sudah digunakan.

3.4.4 Implementasi Router

Router ditempatkan pada file app/core/Router.php. Router memetakan endpoint mahasiswa ke kelas MahasiswaController, kemudian menentukan fungsi yang dijalankan berdasarkan HTTP method.

Pemetaan metode yang diterapkan adalah GET ke fungsi index(), POST ke fungsi store(), PUT ke fungsi update(), dan DELETE ke fungsi destroy(). Apabila endpoint tidak ditemukan, sistem memberikan respons dengan status HTTP 404.

3.4.5 Implementasi Front Controller

Front controller terdapat pada file public/index.php. File ini memuat autoloader dan menjalankan router untuk memproses permintaan yang masuk. File .htaccess digunakan untuk mengarahkan permintaan yang bukan file atau direktori fisik ke index.php.

Dengan mekanisme tersebut, permintaan API dapat diproses melalui satu pintu masuk dan diarahkan sesuai endpoint yang diminta.

3.5 Alur Kerja API

Alur kerja API CRUD pada praktikum ini adalah sebagai berikut.

1. Klien mengirimkan permintaan HTTP ke endpoint /public/mahasiswa.
2. Permintaan diterima oleh file public/index.php.
3. Autoloader memuat kelas yang dibutuhkan.
4. Router memeriksa endpoint dan metode HTTP.
5. Controller memvalidasi permintaan dan memanggil model.
6. Model menjalankan query ke database menggunakan PDO.
7. Controller mengirimkan hasil pemrosesan kepada klien dalam format JSON.

BAB 4. HASIL DAN PEMBAHASAN

4.1 Pengujian GET Seluruh Data Mahasiswa

Pengujian GET dilakukan dengan mengakses endpoint mahasiswa tanpa menyertakan parameter ID. Pengujian ini bertujuan untuk memastikan API dapat mengambil seluruh data dari tabel mahasiswa.

Hasil pengujian menunjukkan bahwa API mengembalikan respons dengan nilai success sebesar true. Data yang ditampilkan terdiri atas tiga mahasiswa, yaitu Andi Pratama, Siti Aminah, dan Budi Santoso. Data diurutkan berdasarkan ID secara menurun.



4.2 Pengujian GET Berdasarkan ID

Pengujian ini dilakukan dengan menambahkan parameter id=1 pada endpoint. Tujuannya adalah memastikan API dapat mengambil data mahasiswa tertentu berdasarkan ID.

Hasil pengujian menunjukkan bahwa API mengembalikan data mahasiswa dengan ID 1, yaitu Budi Santoso dengan NIM 260170001 dan program studi Teknik Informatika. Respons tersebut menunjukkan bahwa proses pencarian data berdasarkan ID berhasil dilakukan.



4.3 Pengujian POST Penambahan Data

Pengujian POST dilakukan untuk memastikan API dapat menambahkan data mahasiswa baru ke dalam database. Data yang digunakan adalah sebagai berikut.

- NIM: 260170004
- Nama: Dewi Lestari
- Program studi: Teknik Informatika

Permintaan dikirim melalui PowerShell menggunakan metode POST dengan format JSON. Hasil pengujian menunjukkan bahwa sistem berhasil menyimpan data dan memberikan pesan Data berhasil ditambahkan. Sistem juga mengembalikan ID data yang baru dibuat, yaitu 4.

```
PS C:\laragon\www\BKPM WORKSHOP SISTEM INFORMASI WEB SERVER\ACARA
 17 API CRUD MVC> $uri = "http://localhost/BKPM%20WORKSHOP%20SIST
EM%20INFORMASI%20WEB%20SERVER/ACARA%2017%20API%20CRUD%20MVC/publi
c/mahasiswa"
PS C:\laragon\www\BKPM WORKSHOP SISTEM INFORMASI WEB SERVER\ACARA
 17 API CRUD MVC> $body = @{
>>     nim  = "260170004"
>>     nama = "Dewi Lestari"
>>     prodi = "Teknik Informatika"
>> } | ConvertTo-Json
>>
>> Invoke-RestMethod -Uri $uri -Method Post -ContentType "applica
tion/json" -Body $body

success message                 data
------- -------                 ----
   True Data berhasil ditambahkan @{id=4}
```

4.4 Pengujian PUT Pembaruan Data

Pengujian PUT dilakukan untuk memastikan API dapat memperbarui data mahasiswa berdasarkan ID. Data yang diperbarui adalah mahasiswa dengan ID 4, yaitu Dewi Lestari.

Nama mahasiswa diubah menjadi Dewi Lestari Update, sedangkan NIM dan program studi tetap menggunakan nilai sebelumnya. Hasil pengujian

menunjukkan bahwa sistem memberikan pesan Data berhasil diperbarui dan mengembalikan data mahasiswa setelah perubahan dilakukan.

```
PS C:\laragon\www\BKPM WORKSHOP SISTEM INFORMASI WEB SERVER\ACARA
 17 API CRUD MVC> $body = @{
>>     nim  = "260170004"
>>     nama = "Dewi Lestari Update"
>>     prodi = "Teknik Informatika"
>> } | ConvertTo-Json
>>
>> Invoke-RestMethod -Uri "$uri`?id=4" -Method Put -ContentType "
application/json" -Body $body

success message                 data
------- -------                 ----
   True Data berhasil diperbarui @{id=4; nim=260170004; nama=...
```

4.5 Pengujian DELETE Penghapusan Data

Pengujian DELETE dilakukan untuk memastikan API dapat menghapus data mahasiswa berdasarkan ID. Data yang digunakan adalah mahasiswa dengan ID 4 yang sebelumnya telah ditambahkan dan diperbarui.

Hasil pengujian menunjukkan bahwa sistem memberikan pesan Data berhasil dihapus. Dengan demikian, fungsi penghapusan data dapat dijalankan melalui metode HTTP DELETE.

```
PS C:\laragon\www\BKPM WORKSHOP SISTEM INFORMASI WEB SERVER\ACARA
 17 API CRUD MVC> Invoke-RestMethod -Uri "$uri`?id=4" -Method Del
ete

success message
------- -------
   True Data berhasil dihapus
```

4.6 Pengujian Validasi NIM Duplikat

Pengujian validasi dilakukan dengan mencoba menambahkan data mahasiswa menggunakan NIM 260170001, yang sudah dimiliki oleh Budi Santoso. Pengujian ini bertujuan untuk memastikan sistem dapat mencegah penggunaan NIM yang sama.

Hasil pengujian menunjukkan bahwa permintaan ditolak dan API mengembalikan respons berikut.

{
 "success": false,
 "message": "NIM sudah digunakan"
}

Respons tersebut menunjukkan bahwa validasi NIM duplikat berjalan sesuai tujuan. Sistem tidak menambahkan data baru karena NIM yang dikirimkan telah digunakan oleh mahasiswa lain. API menggunakan status HTTP 409 Conflict untuk menandai konflik data tersebut.

```
PS C:\laragon\www\BKPM WORKSHOP SISTEM INFORMASI WEB SERVER\ACARA
 17 API CRUD MVC> $body = @{
>>     nim  = "260170001"
>>     nama = "Mahasiswa Duplikat"
>>     prodi = "Teknik Informatika"
>> } | ConvertTo-Json
>>
>> try {
>>     Invoke-RestMethod -Uri $uri -Method Post -ContentType "app
lication/json" -Body $body
>> } catch {
>>     $_.ErrorDetails.Message
>> }




{"success":false,"message":"NIM sudah digunakan"}
```

4.7 Ringkasan Hasil Pengujian

Berdasarkan pengujian yang dilakukan, diperoleh ringkasan sebagai berikut.

No.	Skenario Pengujian	Hasil yang Diharapkan	Hasil
1	GET seluruh data	Seluruh data mahasiswa ditampilkan	Berhasil
2	GET berdasarkan ID	Data sesuai ID ditampilkan	Berhasil

3	POST data mahasiswa	Data baru berhasil ditambahkan	Berhasil
4	PUT data mahasiswa	Data berhasil diperbarui	Berhasil
5	DELETE data mahasiswa	Data berhasil dihapus	Berhasil
6	POST NIM duplikat	Data ditolak dengan pesan kesalahan	Berhasil

Hasil pengujian menunjukkan bahwa seluruh skenario utama API CRUD telah berjalan sesuai harapan. Sistem dapat mengelola data mahasiswa dan menolak penambahan data yang menggunakan NIM duplikat.

BAB 5. PENUTUP

5.1 Kesimpulan

Berdasarkan praktikum Acara 17 yang telah dilaksanakan, dapat disimpulkan bahwa API CRUD menggunakan PHP Native berhasil diimplementasikan dengan menerapkan konsep Object-Oriented Programming (OOP) dan pola arsitektur Model-View-Controller (MVC). Aplikasi menggunakan PDO dan prepared statement untuk mengakses database MySQL serta menerapkan autoloader, front controller, dan router untuk mengatur proses permintaan API.

Fungsi utama yang berhasil diimplementasikan meliputi GET untuk membaca data, POST untuk menambahkan data, PUT untuk memperbarui data, dan DELETE untuk menghapus data. Pengujian melalui browser dan PowerShell menunjukkan bahwa seluruh operasi tersebut berjalan sesuai harapan. Selain itu, validasi NIM duplikat berhasil mencegah penambahan data dengan NIM yang sudah terdaftar.

Dengan demikian, tujuan praktikum untuk membangun dan menguji API CRUD menggunakan PHP Native, OOP, dan MVC telah tercapai.

5.2 Saran

Adapun saran untuk pengembangan selanjutnya adalah sebagai berikut.

1. Menambahkan dokumentasi endpoint yang menjelaskan metode HTTP, parameter, format request, dan respons API.
2. Mengembangkan validasi input agar lebih lengkap, termasuk panjang dan format NIM.
3. Menambahkan autentikasi dan otorisasi apabila API digunakan oleh banyak pengguna.
4. Mengembangkan endpoint untuk sumber daya lain sesuai kebutuhan aplikasi.
5. Mempertahankan penggunaan prepared statement dan penanganan kesalahan agar keamanan serta keandalan aplikasi tetap terjaga.